

投球機 (Ball Machine)

馬蒂娜發明了一台投球機，用來娛樂 IOI 的參賽者。投球機的內部結構如下：

- 投球機共有 N 個節點，編號從 0 到 $N - 1$ 。節點 $N - 1$ 是根節點。
- 對於每個節點 u （其中 $0 \leq u < N - 1$ ）， u 的父節點是 $P[u]$ （其中 $P[u] > u$ ）；節點 u 則稱為 $P[u]$ 的子節點。根節點沒有父節點。沒有子節點的節點稱為葉節點。你並不知道 N 的值，也不知道任意節點 u 的父節點 $P[u]$ 的編號。但是馬蒂娜會告訴你 M 的值，此為葉節點的總數；葉節點的編號為 $0, 1, \dots, M - 1$ 。你的任務是透由一連串的操作，決定出投球機的內部結構。

投球機的每個節點至多可以有一顆球，每顆球上都有一個非負整數的數值。我們說一個節點的值是 x 如果這個節點上有值為 x 的球。我們稱一個節點上有球如果它有被放球；否則稱為空的。一開始所有的節點都是空的。

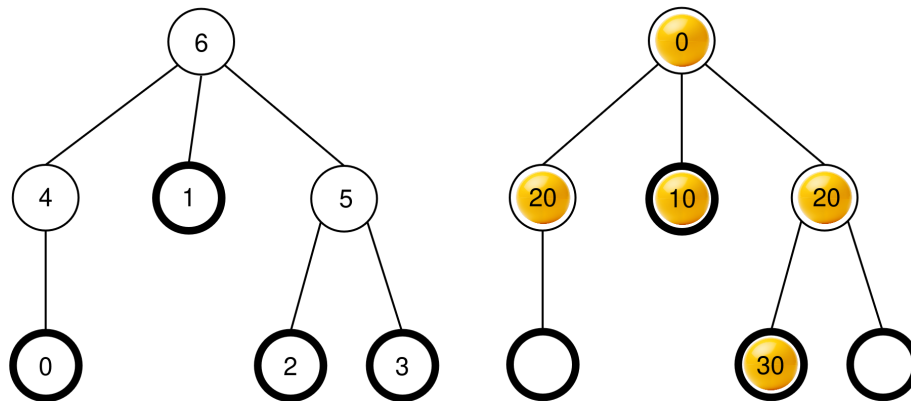
你可以進行下面兩種操作：

- **insert(U, X)**：嘗試將一顆球上寫數字 X 的球塞到葉節點 U 上。
 - 如果葉節點 U 有球，此操作將回傳 `false` 並且不會真的將球塞進去。
 - 如果葉節點 U 上沒球，這顆球會被塞進節點 U 。之後，當球所在的節點的父節點如果是空的，那麼這顆球就會往它的父節點移動，此動作會重複發生；移動停止的條件是當這顆球抵達根節點，或是這顆球所在的節點的父節點上有別的球。此時，此操作將回傳 `true`。
- **collect()**：當根節點上是空的（意謂著投球機是空的），`collect` 將回傳空陣列。否則，下面描述的遞迴程序 `traverse` 會蒐集投球機上所有球的數值，並將其存入陣列 S 中。一開始，陣列 S 是空的，並且從 `traverse( $N - 1$ )` 開始執行。

```
traverse(u):  
    將 u 上的球的數字加入到 S 的末端  
    令 c[u] 為 u 之所有有球的子節點形成的串列  
    將 c[u] 依照節點上的球的數字從小到大排序（如果節點上有相同的數字，任意順序皆可）  
    對於 c[u] 中的每個節點 v:  
        執行 traverse(v)
```

當此程序結束時，投球機上所有的球都會被移除，並且 `collect` 回傳陣列 S 。

舉例來說，假設投球機上有 $N = 7$ 個節點，且其父節點陣列 $P = [4, 6, 5, 5, 6, 6]$ 。左圖為空的投球機，節點上有編號；右圖為經過一連串 `insert` 操作後，可能的一種狀態（操作的方式見範例）。



當 `collect()` 被呼叫，此時根節點（編號 6）上有球，因此 S 被初始成 $S = []$ ，並且 `traverse(6)` 被呼叫。

traverse(6)：節點 6 上有數字 0；將此數字加入 S 末端，得到 $S = [0]$ 。節點 6 之有球的子節點編號為 4、1、5，他們的數字依序為 20、10、20。將它們排序，得到 $c[6] = [1, 5, 4]$ （注意， $c[6] = [1, 4, 5]$ 亦可，因為節點 4 與 5 的數字相同）。此程序接著對排序後的 $c[6]$ 中的每個節點編號進行呼叫。

- **traverse(1)**：節點 1 上有數字 10；將此數字加入 S 末端，得到 $S = [0, 10]$ 。節點 1 下沒有有球的子節點，因此此次呼叫結束。
- **traverse(5)**：節點 5 上有數字 20；將此數字加入 S 末端，得到 $S = [0, 10, 20]$ 。節點 5 下有一個子節點有球（編號為 2），因此 $c[5] = [2]$ ，並且此程序接著呼叫 `traverse(2)`。
 - **traverse(2)**：節點 2 的數字為 30；將其加入 S 末端，得到 $S = [0, 10, 20, 30]$ 。節點 2 沒有子節點，因此此次呼叫結束。
 - 此次執行回溯到 `traverse(5)`，由於 $c[5]$ 所有的節點都被處理完成，因此此次呼叫結束。
- **traverse(4)**：節點 4 上有數字 20；將其加入 S 末端，得到 $S = [0, 10, 20, 30, 20]$ 。注意，節點 4 下沒有有球的子節點，因此此次呼叫結束。

所有的呼叫都結束，並且沒有節點需要處理。之後，投球機所有的球都被清空，並且 `collect` 回傳陣列 $S = [0, 10, 20, 30, 20]$ 。

你的任務要確定節點總數 N ，並且回報描述投球機的內部結構的父節點陣列 $R = [R[0], R[1], \dots, R[N-2]]$ ，但允許不是葉節點與根節點的其他節點的編號可以重排。正式來說，回報的陣列 R 會被當作是正確的，如果存在排列 $L[u]$ （其中 $0 \leq L[u] < N$ ），滿足對於所有的節點 u ：

- $L[u] = u$ 對於所有 $0 \leq u < M$ 及 $u = N-1$ ，且
- $L[P[u]] = R[L[u]]$ 對於所有 $0 \leq u < N-1$ 。

這裡並沒有要求 $R[u] > u$ 。當你回報解答時，投球機必須是空的。

令 K 為 `collect` 操作被呼叫的次數， B 為所有 `insert` 呼叫中球上的數字的最大值。令 $C = K + B$ 。那麼， C 值不能超過 1000。你在某些子任務的分數與 C 值有關。

實作細節

您應該實作下列程序。

```
std::vector<int> find_structure(int M)
```

- M ：投球機的葉節點總數。
- 對於每筆測資，此程序僅會被呼叫一次。
- 此程序必須回傳長度為 $N - 1$ 之陣列 $R = [R[0], R[1], \dots, R[N - 2]]$ ，此為正確的父節點陣列。

為了與投球機互動，你的程序可以呼叫下列兩個程序。

第一個程序是：

```
bool insert(int U, int X)
```

- U ：嘗試塞入球的葉節點的編號。條件 $0 \leq U < M$ 必須被滿足。
- X ：球上的數字。條件 $0 \leq X \leq 1000$ 必須被滿足。
- 此程序回傳 `true` 如果球有被成功塞入；否則回傳 `false` 如果葉節點 U 上原本就有球。
- 此程序至多被呼叫 500 000 次。

第二個程序為：

```
std::vector<int> collect()
```

- 蒐集投球機上所有球的數字、然後清空球，最後回傳蒐集到的陣列 S 。

你的程式的執行過程中，如果 C 值超過 1000，那麼你的解會收到判決 `Output isn't correct: Too many resources used`。

投球機的結構在呼叫 `find_structure` 時就固定不變。評分程式的執行是**確定性的 (deterministic)**，這意謂當你執行兩次相同的操作，呼叫 `collect` 的回傳陣列內容將會完全相同。

限制條件

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

子任務

子任務	分數	額外限制條件
1	5	根節點恰好有 M 個子節點。
2	10	$M \leq 3$
3	25	$N \leq 200$ 且 $M \leq 45$
4	60	無額外限制。

在子任務 3 與 4，你的得分與 C 值有關，如下描述。

子任務 3 (25 分)

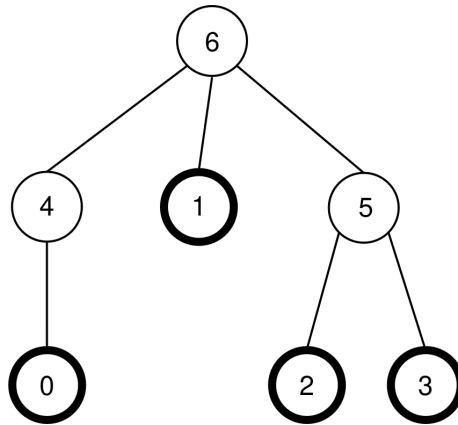
範圍	分數
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

子任務 4 (60 分)

範圍	分數
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

範例

考慮與任務描述中相同的投球機，亦即 $N = 7$ 、 $M = 4$ 且 $P = [4, 6, 5, 5, 6, 6]$ 。投球機的內部狀態再次描述如下：



評分程式進行下列呼叫：

```
find_structure(4)
```

此程序將會進行下列呼叫：

1. **insert(0, 0)**：葉節點 0 是空的，因此有數字 0 的球被塞入節點 0。節點 $P[0] = 4$ 是空的，因此球移到節點 4。節點 $P[4] = 6$ 也是空的，因此球會移到節點 6。由於節點 6 是根節點，故移動結束。此次呼叫回傳 `true`。
2. **insert(3, 20)**：葉節點 3 是空的，因此有數字 20 的球被塞入節點 3。節點 $P[3] = 5$ 是空的，因此球移動到節點 5。因為節點 $P[5] = 6$ 上有球，故移動結束。此次呼叫回傳 `true`。
3. **insert(1, 10)**：葉節點 1 是空的，因此有數字 10 的球被塞入節點 1。因為節點 $P[1] = 6$ 上有球，因此球停留在節點 1。此次呼叫回傳 `true`。
4. **insert(2, 30)**：葉節點 2 是空的，因此有數字 30 的球被塞入節點 2。因為節點 $P[2] = 5$ 上有球，球停在節點 2。此次呼叫回傳 `true`。
5. **insert(1, 25)**：葉節點 1 上有球，因此球不會被塞入節點 1。此次呼叫回傳 `false`。
6. **insert(0, 20)**：葉節點 0 是空的，因此有數字 20 的球被塞入節點 0。節點 $P[0] = 4$ 是空的，因此球移到節點 4。因為 $P[4] = 6$ 上有球，故移動結束。此次呼叫回傳 `true`。

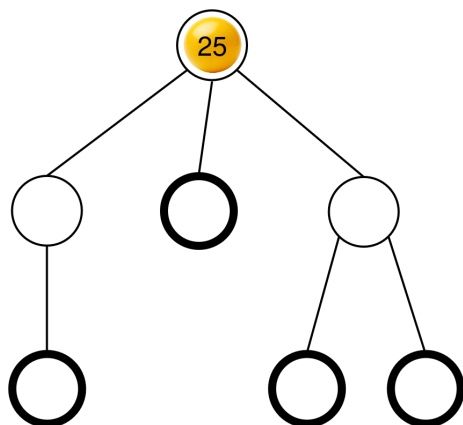
最終球的狀態已經在任務描述中陳述。

接下來，此程序可以呼叫：

```
collect()
```

此次呼叫的過程，已經在任務描述中陳述，因此得到回傳陣列 $S = [0, 10, 20, 30, 20]$ 。到目前為止，投球機再度清空。

接下來，此程序可以呼叫 **insert(2, 25)**。葉節點 2 是空的，因此有數字 25 的球被塞入葉節點 2。球之後被移到節點 5，接著移到節點 6，然後停止。此次呼叫回傳 `true`。此時球的狀態描述如下圖：



最後，此程序呼叫 `collect()`，得到回傳值 $S = [25]$ 並且清空投球機上的球。

`find_structure` 可能回傳兩種正確的父節點陣列： $R = P = [4, 6, 5, 5, 6, 6]$ （對應排列 $L = [0, 1, 2, 3, 4, 5, 6]$ ），以及 $R = [5, 6, 4, 4, 6, 6]$ （對應排列 $L = [0, 1, 2, 3, 5, 4, 6]$ ）。此範例中， $K = 2$ 且 $B = 30$ ，因此 $C = 32$ 。

範例評分程式

輸入格式：

```
N M
P[0] P[1] P[2] ... P[N-2]
```

輸出格式：

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

此處， Q 為 `find_structure` 回傳的陣列 R 的長度。